



EVA: Accelerating LLM Decoding via an Efficient Vector Quantization Architecture

Bowen Duan^{†*}, Cong Guo^{†*§}, Chiyue Wei[†], Haoxuan Shan[†], Yuzhe Fu[†], Xinhua Chen[†],
Yifan Xu[†], Ziyue Zhang[†], Changchun Zhou[†], Hai Li[†], Yiran Chen[†]

[†]Duke University, USA ^{*}Equal contribution [§]Corresponding author
{bowen.duan, cong.guo, hai.li, yiran.chen}@duke.edu

Abstract—Large Language Models (LLMs) have achieved impressive performance across diverse domains but remain inefficient during the autoregressive decoding phase. Unlike the prefill stage, which employs compute-bound GEMM operations, decoding executes a sequence of small GEMV-like computations that are memory-bound and underutilize modern accelerators. Weight-only vector quantization (VQ) has emerged as an effective compression technique that clusters model weights into a shared *codebook* and replaces the original weight matrix with low-precision *indices*, enabling 2-bit-level weight compression. While this approach substantially reduces model size and memory bandwidth, it still suffers from two critical inefficiencies: the low utilization of GEMV computation and frequent memory conflicts during codebook lookups.

This paper presents *EVA*, an efficient vector-quantization-based architecture that addresses both computational and memory bottlenecks in LLM decoding. *EVA* builds on a simple yet effective insight that combines input-codebook computation with conflict-free memory access. Instead of reconstructing quantized weights from indices, *EVA* directly performs dot products between input vectors and the weight codebook, transforming LLM decoding from GEMV to GEMM computation. It then performs structured lookups from an intermediate output buffer, eliminating memory bank conflicts. We further design a hardware-software co-optimized architecture specialized for LLM decoding while remaining compatible with conventional prefill execution. Evaluations show that *EVA* achieves up to $11.17\times$ speedup and $7.17\times$ higher energy efficiency compared with the SOTA lookup-based architecture, while preserving arithmetic precision after vector quantization. Our code is available at <https://github.com/dbw6/Eva.git>.

Index Terms—Large Language Models, Vector Quantization, LLM Decoding, AI Accelerator, Hardware-Software Co-design.

I. INTRODUCTION

Large Language Models (LLMs) [2], [14], [38], [50], [56] have achieved remarkable success across a wide range of domains, from natural language understanding [13], [46] to code generation [17], [55], [63]. However, the inference efficiency of LLMs remains a critical challenge, particularly in the *decoding phase* of the autoregressive (AR) mechanism [60]. During inference, computation is divided into two distinct phases: *prefill* and *decoding*. The prefill phase processes the entire input sequence and can be efficiently executed as large-scale General Matrix Multiplication (GEMM) operations. In contrast, the decoding phase generates one token at a time, resulting in a sequence of small-scale General Matrix-Vector Multiplication-like (GEMV-like) operations. As illustrated in

Fig. 1 (a), this fundamental difference in computational granularity makes the decoding phase significantly less efficient.

Quantization has become an essential technique for efficient LLM inference [19], [21], [23], [26], [37], [39], [45]. Conventional quantization methods [24], [26], [31] typically apply low-precision formats to both weights and activations, enabling reduced-precision computation and lower memory access cost. Recent frameworks primarily focus on *weight-only quantization* [16], [33], [61], [62], such as AWQ [37] and SqueezeLLM [32]. These methods compress model weights into low-bit representations (e.g., 4-bit) while retaining high-precision computation (e.g., FP16) for activations. Weight-only quantization directly targets the *decoding* bottleneck of LLMs, where activations are lightweight but weight tensors dominate memory traffic. Although activations remain in high precision to preserve model accuracy, weight-only quantization achieves nearly linear speedup in the decoding stage as the weight precision decreases [37]. This improvement arises because LLM decoding is inherently memory-bound, and quantization effectively mitigates the bandwidth bottleneck caused by frequent weight loading.

Pushing this trend further leads to *codebook-based compression* [6], [15], [22], where weight tensors are represented using lookup tables (LUTs) instead of arithmetic quantization functions. Unlike integer or floating-point quantization, lookup-based quantization provides higher flexibility and better fidelity by enabling non-uniform representations. Recently, **Vector Quantization (VQ)** [15], [18], [25], [40], [58] has emerged as a pivotal technique that extends codebooks from single-element (scalar) to multi-element (vector) representations, allowing each codebook entry to encode multiple weight elements (e.g., 4 or 8). Compared with arithmetic quantization methods such as AWQ [37], which are typically limited to 4-bit precision, VQ pushes this limit further to 2-bit quantization while maintaining high accuracy, demonstrating superior algorithmic efficiency. As illustrated in Fig. 1 (b), an example with a vector size of $d = 4$ partitions the weight matrix into 4-element vectors, which are then mapped to a compact weight index (WI) matrix and a weight codebook (WC). This multi-dimensional compression enhances quantization expressiveness and achieves state-of-the-art accuracy-compression trade-offs while significantly reducing model size.

Despite its effectiveness in model compression, conventional VQ offers **no speedup** on current hardware accelera-

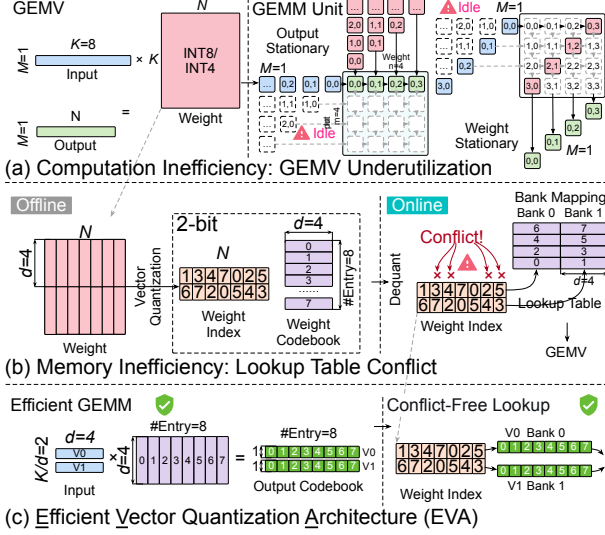


Fig. 1. Motivation of this work. (a) Conventional GEMV suffers from poor utilization of compute units during LLM decoding. (b) Vector quantization introduces irregular and conflicting memory accesses during online lookup. (c) EVA eliminates lookup conflicts and reformulates LLM decoding into GEMM-style computation, enabling efficient execution on modern accelerators.

tors (e.g., GPUs); in fact, it can even be slower than FP16 inference [41]. This limitation arises from two fundamental challenges:

(1) **Compute inefficiency (Fig. 1 (a)).** While VQ effectively reduces memory bandwidth requirements, it does not alter the computational structure of the decoding phase, which remains a sequence of small-scale GEMV operations. Such matrix-vector computation is inherently inefficient on modern accelerators optimized for large GEMM workloads, leading to low parallelism and poor utilization. This inefficiency is not unique to VQ but is a general limitation of weight-only quantization in LLM decoding [1], [49].

(2) **Memory inefficiency (Fig. 1 (b)).** Even after compressed weights are loaded into on-chip memory, the ensuing *dequantization* via codebook lookup often suffers from severe memory access conflicts. These conflicts stem from the irregular and uncoalesced indexing patterns inherent to LUT-based quantization, which lead to serialization, limited memory throughput, and reduced cache efficiency. For example, as shown on the right side of Fig. 1 (b), indices 5 and 3 attempt to access the same memory bank simultaneously, resulting in a memory conflict and serialized access. This bottleneck fundamentally limits their attainable speedup, regardless of their impressive compression ratios.

To overcome these challenges, we propose *EVA*, an **Efficient VQ-based Architecture** that addresses both computational and memory inefficiencies in the LLM decoding phase. *EVA* is built on a simple yet effective *aha insight*, consisting of two main steps that jointly restructure computation and remove memory conflicts.

Step 1: From weight decoding to codebook dot product. Since each vector in the VQ-quantized weight matrix

originates from the codebook, explicit reconstruction is unnecessary. As shown on the left side of Fig. 1 (c), instead of decoding the weight indices (WIs) on-chip to reconstruct the full weight matrix, we directly compute dot products between the input vectors and the weight codebook (WC), generating intermediate results that collectively form an *output codebook* (OC). To support this operation, the input is partitioned into multiple vectors (e.g., v_0 and v_1), each multiplied by the WC to produce corresponding intermediate results that together constitute the OC.

Step 2: Conflict-free lookup from the output codebook.

As shown on the right side of Fig. 1 (c), the final outputs are obtained by performing lookup operations on the OC using the WI matrix. Unlike conventional VQ, where lookups in the WC frequently cause memory bank conflicts, this formulation eliminates such conflicts entirely. During OC computation, matrix multiplication implicitly distributes OC elements across different banks, as each bank stores results derived from distinct input vectors. For example, when indices such as 5 and 3 in the rightmost column of the WI matrix are accessed simultaneously, they map to different banks (corresponding to v_0 and v_1), enabling fully parallel and conflict-free access.

Advantages. This approach improves both computational and memory efficiency. (1) **Computation.** The LLM decoding process is effectively **reformulated from GEMV to GEMM**, increasing arithmetic intensity and parallelism. Since the codebook is small (e.g., 256 entries) while N is large (e.g., 4096), the overall computation cost is significantly reduced. (2) **Memory.** Compared with conventional VQ, accessing the same number of weight indices does not require additional memory banks. At the same time, the bandwidth requirement is reduced from reading $d = 4$ weight elements per access to reading a single OC element, and memory conflicts are completely eliminated. These optimizations make *EVA* efficient in both computation and memory utilization.

In summary, *EVA* transforms the inefficient VQ decoding process into a conflict-free GEMM-style computation. We further design a dedicated hardware architecture that implements this reformulation to achieve high utilization and scalability based on a simple yet effective insight.

Specifically, *EVA* makes the following contributions:

- We propose a **codebook-driven GEMM formulation** that replaces conventional weight decoding with direct dot products between input vectors and the weight codebook. This formulation transforms memory-bound GEMV operations into compute-efficient GEMM computations, significantly improving utilization.
- We develop an **output-codebook lookup mechanism** that reorganizes memory access into a conflict-free structure. By distributing output codebook entries across memory banks, *EVA* eliminates lookup conflicts and reduces bandwidth demand.
- We design and implement a **hardware-software co-optimized architecture** tailored for efficient LLM decoding while maintaining compatibility with conventional accelerator execution in the prefill stage.

Extensive experiments demonstrate that *EVA* achieves up to $11.17\times$ speedup and $7.17\times$ improvement in energy efficiency over the state-of-the-art lookup-based baseline, FIGLUT, while preserving arithmetic precision after vector quantization.

II. BACKGROUND AND MOTIVATION

A. LLM Decoding: Computational Inefficiency

The core operation in transformer-based Large Language Models (LLMs) [28], [56] is matrix multiplication, expressed as $\mathbf{Y} = \mathbf{X}\mathbf{W}$. Here, $\mathbf{X} \in \mathbb{R}^{M \times K}$ denotes the input matrix, and $\mathbf{W} \in \mathbb{R}^{K \times N}$ represents the projection or feed-forward weight matrix. Autoregressive LLMs [60] are typically executed in two distinct phases: *prefill* and *decoding*. During the prefill phase, the model processes multiple tokens from several sequences simultaneously [72]. As a result, all three dimensions (M , K , and N) of the matrix multiplication are large, enabling General Matrix Multiplication (GEMM). This configuration allows GPUs and other accelerators to exploit extensive data reuse, thereby achieving high computational efficiency [7], [30], [49].

In contrast, during the *decoding* phase, the model generates tokens sequentially, one at a time. Consequently, the total number of rows is reduced to $M = 1$, turning the matrix multiplication $\mathbf{X}\mathbf{W}$ from a large-scale GEMM operation into a small-scale General Matrix–Vector Multiplication (GEMV) operation. As shown in Fig. 1 (a), performing GEMV on accelerators optimized for GEMM is highly inefficient for two primary reasons. First, when M is small, only a narrow portion of the GEMM unit is active, while most processing element (PE) lanes remain idle.

Second, GEMV inherently suffers from low arithmetic intensity because one dimension of the matrix cannot be reused across computations. During decoding, activations are small, but the weight matrix dominates the memory footprint and must be repeatedly fetched for each generated token without reuse. As a result, GEMV operations become memory-bound, with the majority of runtime dominated by weight loading rather than computation. This inefficiency has motivated extensive research on compressing the weight matrices during the decoding process to reduce memory bandwidth demand and improve overall inference efficiency [37], [39].

GEMVs occur only in the decode phase with a batch size of 1. In this article, we will refer to both strict GEMVs and GEMV-like GEMMs from small batches, as they both cause array underutilization.

B. Quantization

Quantization methods for LLMs can be broadly categorized into two classes: **analytic quantization** and **non-analytic quantization**. As illustrated in Fig. 2, the key distinction lies in whether the quantization function can be explicitly represented by arithmetic operations (e.g., rounding, scaling).

Analytic quantization. As shown in Fig. 2 (a), analytic quantization refers to schemes with explicit mathematical formulations that map high-precision values (e.g., FP32, FP16) to low-bit values (e.g., INT4/8, FP4/8) through simple operations

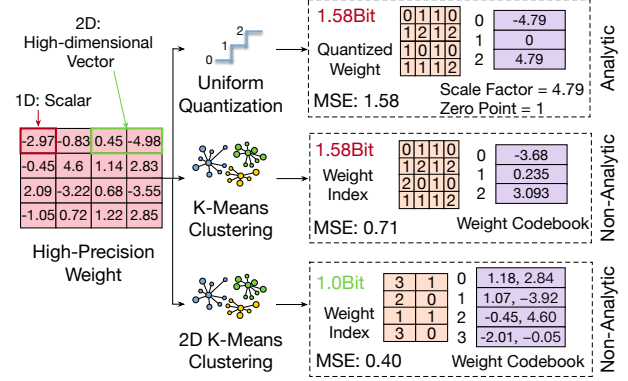


Fig. 2. Comparison of quantization schemes: (a) uniform quantization (analytic); (b) K-means (1D vector) quantization (non-analytic); (c) 2D vector quantization (non-analytic).

such as linear scaling and rounding. Both activations and weights can be quantized in this way, and such approaches are widely adopted for compute-bound workloads where reduced-precision arithmetic directly lowers FLOPs [31], [70].

However, the decoding phase of LLM inference is inherently *memory-bound*, rather than compute-bound. Therefore, modern systems typically employ **weight-only quantization** [16], [37], where weights are stored in low-bit representations (e.g., 4-bit), while activations remain in high precision (e.g., FP16). This strategy effectively reduces memory bandwidth requirements and achieves near-linear speedups proportional to the reduction in data transfer volume.

Nonetheless, the computation itself still operates in high precision. Thus, this approach functions primarily as a *compression technique* rather than a low-precision computation method. Because of its simple and analytic nature, decoding from such quantized weights is efficient: element-wise dequantization can be performed *in-place* without additional dependencies. Consequently, weight-only analytic quantization has been widely adopted in decoding-optimized frameworks such as AWQ [37] and GPTQ [16].

Despite its practicality, analytic quantization offers limited representational flexibility. Its quantization error remains relatively high under aggressive compression. More importantly, **it cannot alter the underlying GEMV computation paradigm**: the decoding process remains a sequence of small, memory-bound matrix–vector multiplications.

Non-analytic quantization. In contrast, non-analytic quantization removes the requirement for closed-form quantization functions. Instead, it directly learns quantization mappings by minimizing a reconstruction loss, typically the mean squared error (MSE), commonly through K-means clustering [6], [15], [22] (Fig. 2 (b)). This approach achieves substantially lower quantization errors than analytic methods and maintains high fidelity even at 2-bit precision. Representative schemes in this category include lookup-table (LUT)-based quantization methods [21], [22], [34] that replace arithmetic dequantization with direct table lookups.

However, this flexibility comes at a cost. Since the mapping

TABLE I
COMPARISON OF REPRESENTATIVE LUT-BASED ACCELERATOR DESIGNS.

Architecture	Codebook Size	Strategy	Replication / Bandwidth Cost
GOBO [73]	8×16 bits (FP16)	Duplication	$768 \times (8 \times 16)$ Registers
LUT TC [44]	8×8 bits (INT8)	Duplication	$2 \times 64 \times (8 \times 8)$ Registers
LUT-DLA [34]	16×16 bits (BF16)	Duplication	$256 \times (16 \times 16)$ Registers
FIGLUT [48]	8×16 bits (FP16)	Broadcast	$16 \times 32 \times (8 \times 16)$ Bandwidth

between original weights and quantized representations is no longer expressible in arithmetic form, decoding requires a *1-to-1 lookup* from codebooks. Such irregular and uncoalesced memory access patterns lead to frequent memory bank conflicts, making the decoding process both expensive and difficult to parallelize, as illustrated in Fig. 1 (b). This limitation has prevented non-analytic quantization from being widely adopted in large-scale inference systems.

Vector Quantization (VQ). Vector Quantization (VQ) extends non-analytic quantization from scalar (1D) K-means to higher-dimensional clustering (multi-D K-means). By encoding multiple elements jointly (e.g., 4D or 8D vectors), VQ captures local structural correlations in weights and substantially reduces quantization loss. As shown in Fig. 2 (c), VQ achieves lower reconstruction error than analytic quantization even with an average of only 1 bit per element, reaching state-of-the-art (SOTA) accuracy-compression trade-offs.

Nevertheless, VQ introduces substantial challenges during decoding. Because each vector code corresponds to multiple weight elements, the effective codebook size increases, which in turn raises memory bandwidth demands and exacerbates access irregularity. This “1-to-many” lookup mechanism makes VQ particularly sensitive to memory access conflicts, posing a fundamental barrier to its efficient deployment on current accelerators.

C. Codebook-based Lookup: Memory Inefficiency

Supporting efficient codebook-based non-analytic quantization has long been a major research objective in both architecture and system communities.

Architectural perspective. From a hardware perspective, prior studies mainly focus on optimizing the lookup table (LUT) design to reduce memory conflicts. Representative works include GOBO [73], FIGLUT [48], LUT Tensor Core [44], and LUT-DLA [34]. As summarized in Tbl. I, these approaches typically rely on either duplicating or broadcasting the codebook across processing elements (PEs) to mitigate bank conflicts. However, both strategies incur substantial hardware and bandwidth overhead. For instance, FIGLUT broadcasts 8×16 -bit data to a full PE column of 32 lanes, resulting in a total bandwidth requirement of $16 \times 32 \times (8 \times 16)$ bits, while LUT-DLA requires $256 \times (16 \times 16)$ registers to store replicated codebook entries. Such design costs significantly constrain scalability and limit the effective codebook size to at most 16 entries.

Software and system perspective. From the algorithmic and system side, prior VQ frameworks primarily focus on improving accuracy while adopting naive software implementations. AQLM [15] and QuiP [58] are implemented in PyTorch

TABLE II
SUMMARY OF NOTATIONS.

Symbol	Description	Value
M	Input (output) height, i.e., number of tokens	1
N	Weight (output) width, i.e., output channel	4096+
K	Input width or weight height for reduction	4096+
d	Vector dimension used in VQ	8
n	Bit-width of each vector index	8
2^n	Number of entries per codebook	256
$V = \frac{K}{d}$	Height of compressed weight index matrix	512+
v	Tile height for grouped computation	32
C	Number of codebooks per layer	2–4
$q = \frac{C \times n}{d}$	Effective quantization bit-width (average)	2–4
$\mathbf{I}$	Weight index (WI) matrix	$[0, 2^n)^{V \times N}$
$\mathbf{B}$	Weight codebook (WC) containing 2^n centroids	$\mathbb{R}^{d \times 2^n}$
$\mathbf{O}$	Output codebook (OC)	$\mathbb{R}^{V \times 2^n}$

and mainly serve as model compression techniques, often slower than FP16 inference. VQ-LLM [41] is the first GPU-based framework to address codebook conflicts; however, due to GPU hardware constraints, it only mitigates conflicts by profiling codebook access frequencies to classify *hot* and *cold* entries. This heuristic approach alleviates but does not eliminate memory contention, leaving the fundamental inefficiency of VQ unresolved.

As a result, despite its superior compression and accuracy, vector quantization remains limited by its poor hardware efficiency and irregular access patterns. In this study, we propose **EVA**, which overcomes both computational inefficiency and memory conflicts through a unified, architecture-aware VQ design.

III. OVERVIEW OF EVA

This section presents the computation flow and architectural overview of EVA, as illustrated in Fig. 3. We first define the key variables and symbols used throughout this study, summarized in Tbl. II.

A. Vector Quantization

GEMV. As illustrated in Fig. 3 (a) ①, the LLM decoding phase performs a sequence of matrix–vector multiplications (GEMV). For each generated token, the computation is expressed as

$$\mathbf{y} = \mathbf{x}\mathbf{W},$$

where $\mathbf{x} \in \mathbb{R}^{1 \times K}$ is the token activation vector and $\mathbf{W} \in \mathbb{R}^{K \times N}$ represents the model weights. During autoregressive generation, $\mathbf{x}$ updates token by token, requiring multiple small GEMV operations executed sequentially. Each operation reads the full weight matrix from off-chip memory but performs limited arithmetic, making the process inherently *memory-bound*. This low arithmetic intensity causes severe underutilization of compute resources on modern accelerators. To alleviate this bottleneck, quantization is commonly applied to reduce the storage and bandwidth requirements of $\mathbf{W}$.

Vector Quantization. As shown in Fig. 3 (a) ②, Vector Quantization (VQ) compresses the weight matrix $\mathbf{W}$ by grouping d consecutive weights along the K dimension into d -dimensional vectors. Each group of d weights forms a small

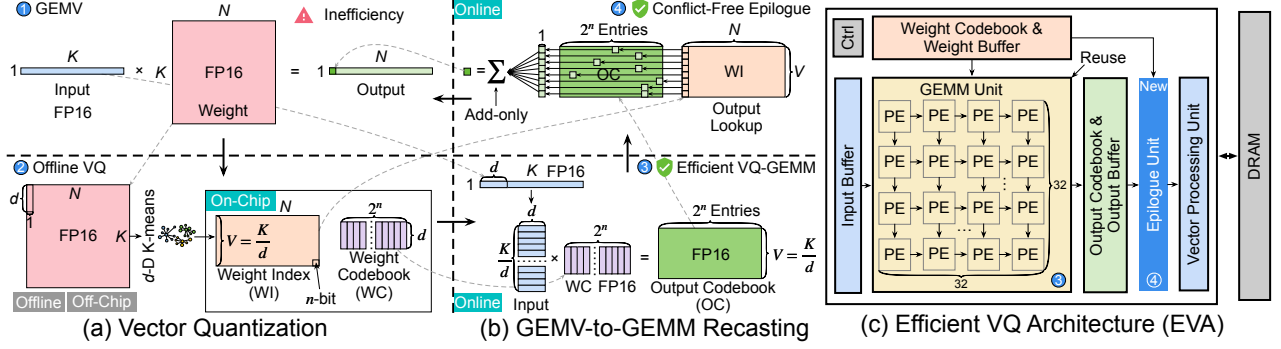


Fig. 3. Overview of the EVA computation flow and architecture.

vector that is replaced by an index referencing a shared weight codebook (WC) **B**. The codebook contains 2^n representative d -dimensional centroids (where n is the index bit-width), each learned from the distribution of weights using k -means clustering [15], [22]. All $K \times N$ weight elements are therefore represented as $V \times N$ indices, where $V = K/d$. The weight index (WI) matrix **I** stores these compact indices, while the codebook **B** stores the learned centroids.

During inference, the quantized model no longer loads the original full-precision weights. Instead, each weight vector is reconstructed on demand by fetching its corresponding centroid from the codebook. This process replaces bandwidth-intensive FP16 weight loading with lightweight index-based lookups, significantly reducing the storage and memory access cost of LLM layers.

In this study, we adopt $d = 8$ for the vector dimension and $n = 8$ for the index bit-width, resulting in a codebook of $2^n = 256$ centroids. This corresponds to an average quantization rate of approximately $\frac{n}{d} = 1$ bit per element for one codebook, which, however, is too low to preserve model accuracy. To achieve higher precision while maintaining compression efficiency, we adopt the additive vector quantization strategy AQLM [15] and use multiple codebooks for hierarchical refinement. Specifically, we employ C codebooks to obtain an effective average quantization precision of $q = \frac{C \times n}{d}$ bits. EVA supports $C = 2, 3$, and 4 codebooks, corresponding to 2-bit, 3-bit, and 4-bit quantization for LLM decoding.

Computation and Memory Inefficiencies. After dequantization, conventional VQ-based methods continue with the GEMV operation as shown in step ① [15], [58]. This offline VQ process significantly reduces memory footprint and data movement. However, it does not change the fundamental LLM decoding computation pattern, since each token still performs a GEMV using the reconstructed weights. Furthermore, the lookup-based reconstruction introduces irregular memory access patterns that cause frequent bank conflicts and degrade parallel efficiency. Compared with 1D k -means quantization, VQ demands higher bandwidth and larger codebooks. These characteristics increase the memory access complexity of VQ, but they also reveal an inherent structural regularity that can be exploited for more efficient computation.

B. Recasting GEMV to GEMM

While conventional VQ increases codebook dimensionality and lookup complexity, its multi-dimensional centroids also expose an opportunity for computation reorganization. By exploiting this property, we reformulate the LLM decoding computation so that the original GEMV can be recast as a GEMM operation, thereby improving compute efficiency. The detailed process is described below.

Dot product between input and centroids. As illustrated in Fig. 3 (b) ③, we first reshape the input token vector $\mathbf{x} \in \mathbb{R}^{1 \times K}$ into a two-dimensional matrix $\mathbf{X} \in \mathbb{R}^{(K/d) \times d}$ by grouping every d consecutive elements into a vector. According to matrix multiplication rules, each d -dimensional row vector of $\mathbf{X}$ performs a dot product with the corresponding d elements from each column of the weight matrix $\mathbf{W} \in \mathbb{R}^{K \times N}$. After applying VQ, these d elements are replaced by the centroids referenced by the weight index (WI) matrix $\mathbf{I} \in [0, 2^n)^{(K/d) \times N}$. Therefore, each input vector interacts with the corresponding centroids across all output channels. This means that, regardless of the output dimension N , the number of possible weight values for each input vector is bounded by the codebook size 2^n .

VQ GEMM. This observation enables a key simplification. We can directly multiply the reshaped input matrix $\mathbf{X}$ with the codebook $\mathbf{B} \in \mathbb{R}^{d \times 2^n}$ to obtain an intermediate result called the output codebook (OC). Each element in the output codebook $\mathbf{O} \in \mathbb{R}^{(K/d) \times 2^n}$ represents the dot product between one input vector and one centroid, which can be reused across multiple output channels. This operation effectively converts the token-level GEMV into a compact GEMM between $\mathbf{X}$ and $\mathbf{B}$:

$$\mathbf{O} = \mathbf{X}\mathbf{B},$$

where $\mathbf{O}$ is the output codebook. This formulation bridges vector quantization and matrix multiplication, paving the way for efficient GEMM-based LLM decoding on modern accelerators.

Advantages. This reformulation provides several key benefits for efficient LLM decoding:

- 1) *Regularized memory access.* Converting LLM decoding into GEMM avoids direct lookups on the weight codebook, which are irregular. The number of lookups is

reduced, and all memory accesses become regular and coalesced.

- 2) *Lower bandwidth demand.* The dot-product operation aggregates every d weights into one value, reducing the lookup bandwidth requirement by a factor of d .
- 3) *Reduced computation.* For LLMs, the output dimension N often exceeds 4096, while the codebook size is only $2^n = 256$. Conventional GEMV requires $1 \times N \times K$ multiply-accumulate operations, whereas VQ-GEMM needs only $\frac{K}{d} \times 2^n \times d = K \times 2^n$ operations, achieving approximately $\frac{N}{2^n} = 16\times$ fewer computations.
- 4) *Higher accelerator utilization.* Recasting LLM decoding into GEMM greatly improves hardware utilization. The matrix dimension in the M direction is no longer fixed at 1, but expanded to $V = \frac{K}{d}$, which is typically greater than 512, enabling efficient use of the accelerator's matrix-multiplication units.

Together, these advantages allow *EVA* to achieve high throughput with regular memory access, lower bandwidth demand, and significantly improved compute efficiency.

C. Epilogue for Lookup and Reduction

After the VQ-GEMM operation, we obtain the intermediate result matrix $\mathbf{O}$, as shown in Fig. 3 (b) ④. In the epilogue stage, two operations are required to reconstruct the final output of LLM decoding.

1) **Lookup using the index matrix.** Each output element is selected from $\mathbf{O}$ according to its corresponding index in the weight index matrix $\mathbf{I}$:

$$\hat{\mathbf{y}} = \text{Lookup}(\mathbf{O}, \mathbf{I}),$$

where $\hat{\mathbf{y}}$ denotes the intermediate partial sum. This lookup is conflict-free because both $\mathbf{I}$ and $\mathbf{O}$ share the same height $V = K/d$. Each row of $\mathbf{O}$ is mapped to a dedicated memory bank, enabling parallel access along the V dimension without conflicts. Compared with conventional VQ methods, accessing the same number of WI entries does not require additional memory banks. However, the bandwidth of each bank is reduced by a factor of d , since each access fetches only one FP16 element instead of an $8d$ centroid with 8 FP16s.

2) **Reduction by accumulation.** After lookup, the selected elements are accumulated to form the final output vector. This add-only reduction requires no multipliers and incurs minimal hardware cost, allowing the epilogue to run efficiently in parallel with the GEMM pipeline.

Advantages. The epilogue is lightweight and hardware-friendly, featuring: (i) add-only operations, (ii) conflict-free parallel access, (iii) reduced bandwidth by a factor of d , and (iv) a simple, efficient pipeline design.

Overall, this stage completes the LLM decoding process with minimal overhead and provides a clean interface between VQ-GEMM computation and final output reconstruction.

D. Efficient VQ Architecture

Finally, Fig. 3 (c) presents the overall *EVA* architecture. Thanks to the structured scheduling of the VQ-GEMM op-

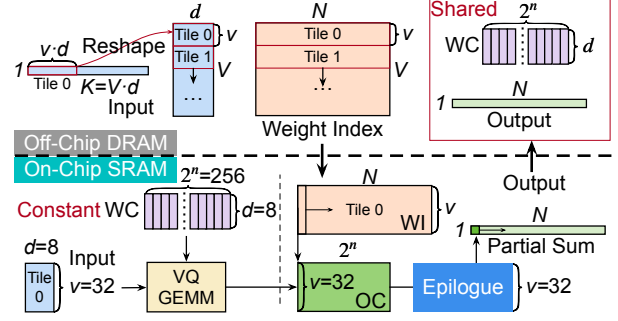


Fig. 4. Tiling strategy of the VQ-GEMM operation in *EVA*. Input and WI tensors are streamed as tiles from off-chip memory, while the WC and output remain stationary on-chip for reuse.

eration, the architecture design remains simple, scalable, and highly generalizable across different accelerators.

First, *EVA* adopts a systolic-array-based computation core, which has been widely used in modern AI accelerators [19], [21], [27]. The proposed VQ-GEMM operation can be directly mapped onto the existing systolic array with minimal modification. While VQ requires FP16 arithmetic for accuracy, the LLM prefill stage can maintain accuracy using only INT8 precision [31], [67]. To support both regimes efficiently, we introduce a reconfigurable PE design that performs one FP16 operation or four INT8 operations per cycle, enabling seamless and high-throughput reuse across FP16 VQ decoding and INT8 prefill computation.

Second, a dedicated epilogue unit is introduced to support VQ-specific post-processing, including the lookup and accumulation operations described earlier. This unit is optimized for conflict-free access and add-only reduction, ensuring low latency and efficient integration with the systolic array pipeline.

These architectural components and their co-optimization strategies will be discussed in detail in the following sections.

IV. VECTOR QUANTIZATION-BASED GEMM

This section introduces the VQ-based GEMM design in *EVA*, including the tiling strategy, the mixed-precision GEMM unit, and its architectural integration with the epilogue stage.

A. Tiling Strategy

Tiling is an essential optimization for GEMM execution because the on-chip memory resources of accelerators are limited. As shown in Fig. 4, the tensors involved in the VQ-GEMM computation, including the input, weight index (WI), weight codebook (WC), and output, are pre-arranged in off-chip DRAM before execution. Among these tensors, only the input and WI matrices are tiled for streaming access, while the WC and output remain stationary on-chip to maximize reuse.

For the input, the matrix is reshaped and divided into tiles of size $v \times d$, where v is set to 32 in our design. Each input tile is loaded into the on-chip buffer for one round of computation. Similarly, the WI tensor is accessed by v rows per iteration, corresponding to $v \times N$ elements. Since $v \times N$ can be large for

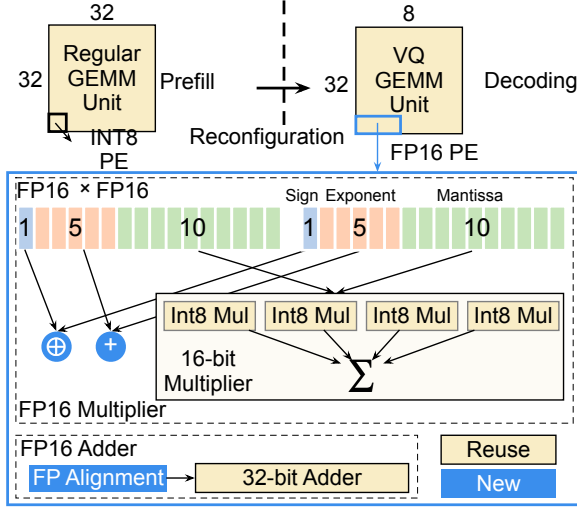


Fig. 5. Reconfigurable GEMM unit in *EVA*. The mixed-precision PE array supports both INT8 prefill and FP16 VQ-GEMM for LLM decoding through precision reconfiguration.

modern LLM layers, WI is streamed into the chip to balance throughput and buffer utilization.

In contrast, both the weight codebook (WC) and the output are shared within a layer and remain stationary in on-chip SRAM. The WC is read-only throughout the layer, while the output tile is progressively updated after each VQ-GEMM iteration and finally written back to DRAM after all partial sums are accumulated.

The on-chip buffers are organized into two regions: (1) one for the input and WC tensors participating in GEMM computation, and (2) another for the WI and output tensors, which are consumed and updated during the epilogue stage (Sec. V).

B. Mixed-precision Processing Element

Although *EVA* is primarily designed for LLM decoding, it must also maintain compatibility with other workloads such as prefill and attention computations. For prefill and attention computations, FP16 precision is often unnecessary, and low-precision INT8 arithmetic is sufficient [31], [67]. Therefore, the base GEMM unit in *EVA* adopts an INT8-based processing element (PE) design, which is widely used in modern accelerators. However, LLM decoding relies on weight-only quantization and FP16 computation to maintain accuracy. Supporting both modes independently would require a separate FP16 multiplier array, resulting in substantial hardware overhead.

To address this challenge, *EVA* reuses the existing INT8 multiply-accumulate (MAC) array to support FP16 operations through lightweight reconfiguration, as illustrated in Fig. 5. An FP16 number consists of a 1-bit sign, a 5-bit exponent, and a 10-bit mantissa. By decomposing each FP16 multiplication into four INT8 multiplications, the PE reconstructs FP16 mantissa multiplication without requiring a dedicated FP16 multiplier. A small 6-bit adder and several XOR gates are

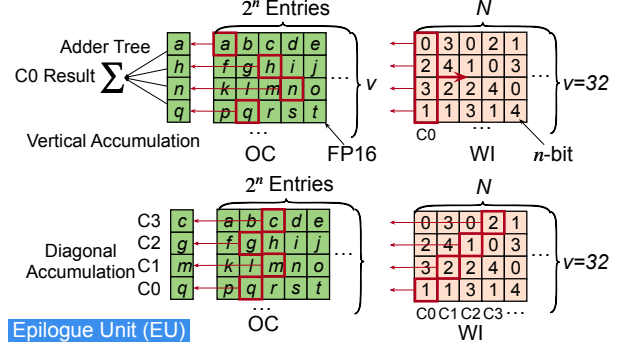


Fig. 6. Epilogue unit (EU) for conflict-free lookup in *EVA*, supporting vertical adder-tree reduction and diagonal accumulation across outputs (C0–C3) with output-level parallelism.

added to handle exponent and sign computation. For FP16 addition, an alignment unit is inserted before accumulation, and the existing INT32 accumulator is reused for the final summation.

This reconfiguration enables FP16 support with minimal hardware modification. A conventional 32×32 INT8 array can be dynamically reconfigured into a 32×8 array for FP16 computation during LLM decoding. This configuration perfectly matches the tiling parameters ($v = 32$, $d = 8$) used in the VQ-GEMM operation, allowing the GEMM unit to achieve optimal utilization and performance. The result is a unified GEMM unit capable of executing both INT8 and FP16 operations with high area efficiency and utilization, enabling *EVA* to flexibly support both prefill and LLM decoding workloads.

V. EPILOGUE UNIT AND PIPELINE

A. Epilogue Unit

The epilogue unit (EU) in *EVA* is designed to be lightweight, parallel, and highly efficient. As shown in Fig. 6, it performs two core operations: conflict-free lookup and add-only accumulation.

Basic Units. Each EU performs a fast n -bit index lookup followed by an add-only reduction. For each tile, $v = 32$ weight indices (WIs) are read in parallel and used to fetch the corresponding FP16 values from the output codebook (OC). The retrieved values are accumulated using one of two schemes: (1) a vertical 32-input adder-tree reduction for single-codebook execution, or (2) a diagonal accumulation scheme that enables output-level parallel reduction across multiple codebooks (e.g., C0–C3). Both designs maintain conflict-free access and high utilization while avoiding multipliers in the epilogue stage.

EU Efficiency. Although each EU contains only 32 adders, each accumulates $d = 8$ dot-product results rather than a single scalar. Thus, the 32 adders collectively process $\frac{32 \times d}{C} = 128$ weight elements and their corresponding dot products, equivalent to conventional VQ decoding. In contrast, traditional designs require conflict-prone memory accesses and costly MAC operations. By using this add-only scheme, *EVA*

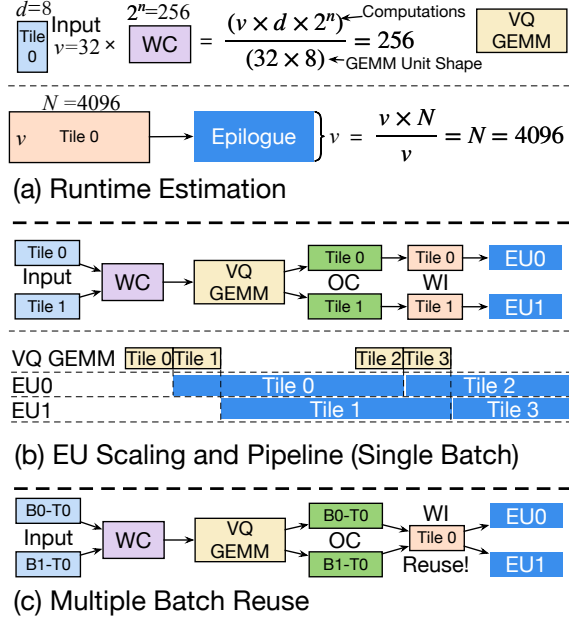


Fig. 7. Execution scheduling of EVA. (a) Runtime estimation showing that the GEMM stage is not the bottleneck. (b) EU scaling pipeline demonstrating GEMM–Epilogue overlap. (c) Multi-batch reuse, where multiple requests share the same weight tiles to reduce bandwidth cost.

achieves the same functionality with lower computation and hardware cost, without sacrificing arithmetic precision.

B. EU Scaling

Fig. 7 (a) illustrates the runtime estimation and pipeline organization of EVA. The execution latency of VQ-GEMM on a 32×8 GEMM unit depends only on the tiling parameters v , d , and 2^n , and is independent of the original matrix width N . For instance, when $v = 32$, $d = 8$, and $2^n = 256$, one VQ-GEMM operation requires only 256 cycles, whereas the EU ($N = 4096$) would take 4096 cycles. In this configuration, GEMM is not the bottleneck; instead, the critical path resides in the epilogue unit (EU).

As shown in Fig. 7 (b), the system overlaps VQ-GEMM computation with epilogue processing to maximize hardware utilization and throughput. The output tiles produced by the GEMM unit are directly consumed by the EUs without off-chip transfers, reducing latency and avoiding bandwidth contention. This concurrent scheduling allows the EU to sustain near-peak utilization across all computation stages.

Consequently, GEMM units become partially idle as the bottleneck shifts from multiplication to addition. Since the EU consists only of adders and requires much lower bandwidth, its scaling cost is minimal. Therefore, overall performance can be improved by simply increasing the number of EUs to better match the GEMM output rate. However, the number of EUs cannot grow indefinitely and must be jointly optimized with respect to model size, VQ configuration, GEMM throughput, and memory bandwidth. We conduct a design-space exploration (DSE) in Sec. VI-B to analyze these trade-offs.

C. Batch Scaling

EVA also efficiently supports multi-batch execution, enabling system-level optimizations such as continuous batching. As shown in Fig. 7 (c), when handling two requests, each with its own Tile 0, the VQ-GEMM stage operates identically to the single-request case. At the EU stage, the Tile 0 results from both requests can reuse the same weight tile, significantly reducing bandwidth consumption. This reuse capability improves overall throughput and efficiency for multi-batch workloads. We further evaluate the effect of batch scaling on EVA’s performance in Sec. VI-E.

VI. EVALUATION

A. Methodology

Accelerator Baselines. We compare EVA with four baseline architectures: Systolic Array (SA) [30], ANT [21], FIGNA [27], and FIGLUT [48]. The Systolic Array [30] optimizes matrix multiplication by enhancing data reuse and reducing external memory bandwidth. ANT [21] introduces a fixed-length data type and an adaptive hardware framework for low-bit quantization. FIGNA [27] converts floating-point activations into integers using pre-alignment, allowing for efficient integer-only computation. FIGLUT [48] replaces FP-INT GEMM with a look-up table architecture that retrieves precomputed partial sums of activations based on weight patterns.

Accelerator Implementation. We develop our own simulator based on the validated open-source simulator, ANT [21]. All baseline accelerators are integrated into the simulator to compare their performance with EVA. We implement the hardware architectures for EVA and all baseline models in Verilog HDL. To ensure a fair comparison, we synthesize all designs using Cadence Genus with the TSMC 28nm technology library, targeting a clock frequency of 500MHz to measure hardware area and power. Additionally, we simulate the SRAM buffer area and power using Cacti 7.0 [3], which is also based on the 28nm process. Finally, the power consumption of DRAM is simulated based on DRAMsim3 [36].

Benchmark. We evaluate EVA with baselines using the LLaMA 1, 2, and 3 models [54], [56], [57], as well as two advanced Mixture-of-Experts (MoE) models: Mixtral-8x7B [29] and Qwen3-30B-A3B [69]. We employ two types of datasets for our benchmarking. First, we use a synthetic dataset with a fixed input length (M) to evaluate model performance during the decoding phase. Second, we assess end-to-end performance in real-world LLM inference scenarios using the Dolly [12], Arxiv Summarization [11], and GSM8K [10] datasets. To simplify our experiments, we run the first Transformer block of each model.

B. Design Space Exploration

We conduct design space exploration on three key vector quantization (VQ) parameters n , C , and d (defined in Tbl. II) and the number of Epilogue Units (EUs) to identify the optimal EVA configuration.

TABLE III
NORMALIZED *EVA*'S LATENCY ON LLAMA-2-7B ACROSS DIFFERENT VQ CONFIGURATIONS.

Algorithm	d	n (2^n)	C	q	N	PE:EU	Norm. Latency
AQLM 2×8	8	8 (256)	2	2	4096	1 : 16	1.00×
AQLM 3×8	8	8 (256)	3	3	4096	1 : 16	1.49 ×
AQLM 2×12	8	12 (4096)	2	3	4096	1 : 1	2.96 ×
AQLM 4×8	8	8 (256)	4	4	4096	1 : 16	1.98 ×
AQLM 1×16	8	16 (65536)	1	2	4096	16 : 1	22.86 ×
GPTVQ-4D	4	8 (256)	1	2	256	1 : 1	4.17 ×
Hypothesized	4	8 (256)	1	2	4096	1 : 16	1.00 ×

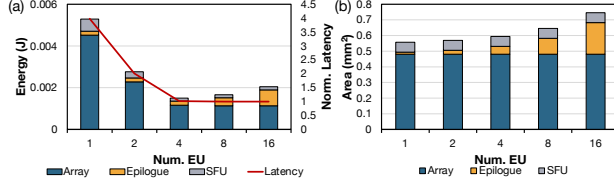


Fig. 8. Design space exploration of *EVA*. (a) Number of Epilogue Units with latency and energy. (b) Number of Epilogue Units with area.

VQ parameters. Tbl. III shows how *EVA*'s latency varies across different VQ configurations. Here, N denotes the minimum number of output channels sharing the same codebook; for AQLM, this corresponds to the linear-layer output dimension ($N \geq 4096$). As shown in the table, when $2^n < N$, latency is approximately proportional to the effective bitwidth $q = \frac{nC}{d}$; thus configurations with the same q exhibit similar performance. After VQ reformulation, computation splits into PE-side multiplications $K \times 2^n$ and EU-side accumulations $K \times N$, yielding the ratio PE:EU = $\frac{2^n}{N}$. For example, when $n = 12$ and $N = 4096$, we have $2^n = N$ (PE:EU = 1:1). In this balanced regime, the systolic array latency exceeds the Epilogue latency, causing *EVA* to become PE-bound and reducing its acceleration benefit. When $2^n > N$, this effect becomes more pronounced; centroid under-utilization introduces “spurious” multiplications (presented in the Sec. VI-F), further increasing latency. Our design-space exploration confirms that $2^n = 256$ ($n = 8$) provides the most practical trade-off between compute efficiency and memory cost.

Number of Epilogue Units. As shown in Fig. 8 (a) and Fig. 8 (b), we fix the DRAM bandwidth at 64 GB/s and evaluate the impact of EU count on latency, energy, and area. Increasing the number of EUs initially reduces both latency and energy, but beyond four EUs, latency no longer decreases. This is because in each cycle the four EUs can process $4 \times v = 128$ weight indexes, which fully utilizes the 64 GB/s DRAM bandwidth at a clock frequency of 500 MHz. Further increasing EU only raises energy consumption. Since each EU only consists of adders, its area overhead remains negligible (3.5%) compared to the PE array when the number of EUs is small.

Architecture configuration. Our exploration indicates that during the decoding stage, the best trade-off is achieved by matching the number of EUs with memory bandwidth. However, higher bandwidth can lead to lower utilization during

TABLE IV
EVA ARCHITECTURAL CONFIGURATION

PE Array Mixed-precision	32×32 INT8, Weight Stationary; 32×8 FP16, Input Stationary.
Epilogue Unit	4×32 -input Adder Tree.
Decoding Stage Tiling	$m = M$; $k = \frac{4 \times v \times d}{M}$; $n = N$.
Prefill Stage Tiling	$m = 1024$; $k = 32$; $n = 1024$.
Quantization: q -bit	$d = 8$; $n = 8$; $C = q$.
Buffer Size: 528KB	16KB Weight Codebook; 256KB Weight; 32KB Input; 192KB Output Codebook; 32KB Output.
DDR4 DRAM	8Gb $\times$ 8 2133R; 16GB/s per channel; 4 channels 64GB/s.

TABLE V
THE PERPLEXITY ON WIKITEXT DATA.

Arch.	SA	ANT	FIGNA	FIGLUT	<i>EVA</i>	FIGNA	<i>EVA</i>	FP16
Algo.	QSERVE	ANT	AWQ	BCQ	AQLM	AWQ	AQLM	FP16
FC Act.	INT8	8bit	FP16	FP16	FP16	FP16	FP16	FP16
FC Wgt.	INT8	8bit	INT4	4bit	4bit	INT2	2bit	FP16
L-2 7B	5.56	5.58	5.60	5.58	5.43	2.2e5	6.69	5.12
L-2 13B	4.95	5.20	4.97	4.96	4.76	1.2e5	5.63	4.57

the prefill phase and increase DRAM power consumption. Additionally, increasing the number of EUs results in a rise in both the architecture's area and buffer size. Therefore, we have chosen to implement four EUs with a memory bandwidth of 64 GB/s. A complete summary of the architectural configuration of *EVA* is presented in Tbl. IV.

C. Model Performance

Comparison of *EVA* and the baselines. We evaluate the LLaMA-2-7B (L-2 7B) and LLaMA-2-13B (L-2 13B) on the WikiText-2 dataset [43], comparing *EVA* with accelerator baselines using their respective state-of-the-art (SOTA) quantization algorithms.

As shown in Tbl. V, systolic arrays adopt INT8 multiply-accumulate (MAC) units with QSERVE [39], ANT applies its dedicated 8-bit format, FIGNA supports FP16 activation and INT4 weight for AWQ [37], FIGLUT uses binary-coding quantization (BCQ) with ShiftAddLLM [71], and *EVA* employs AQLM [15], [42] with parameters $d = 8, n = 8$. All methods maintain Attention layers in FP16 precision.

Tbl. V presents the perplexity results, where lower perplexity indicates better performance. At higher quantization precision (e.g., 8-bit or 4-bit weight-only), all algorithms achieve comparable accuracy with minimal degradation from

TABLE VI
ACCURACY(%) COMPARISON OF LLAMA-2-7B ON DOWNSTREAM BENCHMARK.

Method	Bits	PIQA	COPA	ARC-E	ARC-C	Winogrande
FP16	16	78.1	87.0	76.4	43.5	69.1
LLM.265 FB	4	78.8	—	73.83	44.1	69.7
LLM.265 VB	4	78.9	—	73.82	45	69.7
<i>EVA</i>	4	77.0	85.0	74.1	41.0	68.5
LLM.265 FB	2	54.3	68.5	29.76	30.5	51.8
LLM.265 VB	2	56.7	68.9	34.52	31.1	52.3
<i>EVA</i>	2	75.9	84.0	71.7	38.6	68.2

TABLE VII
ACCURACY (%) COMPARISON OF MIXTURE OF EXPERTS (MoE)
MODELS.

Method	Bits	ARC-C	ARC-E	PIQA	BoolQ	Winogrande
Mixtral-8x7B						
FP16	16	59.81	83.54	83.73	85.26	76.56
AWQ	4	58.87	82.58	82.97	84.34	76.24
AQLM-2×16	4	54.61	83.12	81.99	–	74.82
AQLM-4×8	4	58.87	83.38	83.51	85.02	76.01
GPTQ	2	27.30	35.44	59.79	52.08	50.83
GPTVQ-4D	2	46.42	65.57	78.13	78.59	71.11
AQLM-1×16	2	47.93	77.68	80.43	–	75.93
AQLM-2×8	2	50.43	78.24	80.69	81.28	71.74
Qwen3-30B-A3B						
FP16	16	62.80	83.75	80.36	88.47	73.88
AWQ	4	61.43	82.66	80.90	88.87	73.01
AQLM-4×8	4	61.26	82.95	80.96	89.02	73.72
AQLM-2×8	2	54.27	76.85	76.93	86.94	69.69

FP16, while *EVA* slightly outperforms others. When precision decreases to 2-bit, weight-only quantization collapses, but the VQ-based method maintains competitive accuracy due to its higher-dimensional clustering.

We further compare *EVA* with the video-codec-based LLM.265 [68] framework on downstream benchmarks, including PIQA [4], COPA [51], ARC-Easy, ARC-Challenge [9], and Winogrande [52]. As shown in Tbl. VI, at higher bitwidths, all methods perform similarly, while only VQ-based *EVA* retains competitive accuracy at 2-bit precision. On average, *EVA* improves accuracy by 19 percentage points over LLM.265 VB at 2-bit across all benchmarks.

***EVA*'s accuracy trade-off.** We compare vector quantization algorithms used in the *EVA* (AQLM [15] and GPTVQ [59]), with weight-only methods on Mixture of Experts (MoE) models (Mixtral-8x7B [29] and Qwen3-30B-A3B [69]). We evaluate the accuracy on various downstream datasets, including ARC-Easy, ARC-Challenge [9], PIQA [4], BoolQ [8], and Winogrande [52].

As shown in Tbl. VII, the 4-bit VQ algorithm achieves nearly lossless compression, with an average accuracy decrease of less than 0.5%. At 2 bits, the AQLM 2×8 configuration shows a 5.3 percentage-point accuracy drop on Mixtral-8x7B, outperforming the non-analytic GPTQ method (32.7% drop). We find no significant accuracy improvements with larger codebook sizes (n) or fewer output channels N at the same bit width. In particular, increasing the number of codebooks C can improve accuracy while keeping n small. Tbl. III and Tbl. VII show that the AQLM 1×16 configuration with lower bit width performs worse than the AQLM 4×8 configuration with higher bit width on both efficiency and accuracy. Thus, using the VQ configuration with $n = 8$ for the *EVA* evaluation is justified.

Overall, Tbl. V, Tbl. VI, and Tbl. VII highlight the robustness of VQ-based quantization at low precision. Importantly, *EVA* is decoupled from specific quantization algorithms: it

TABLE VIII
THE AREA, POWER, THROUGHPUT, AND EFFICIENCY COMPARISON OF *EVA*
AND BASELINE ACCELERATORS (28NM, 500 MHZ).

Architecture	SA	ANT	FIGNA	FIGLUT	<i>EVA</i>
Area (mm ²)	1.256	1.472	1.211	1.582	1.414
On-chip Power (W)	1.647	2.741	2.602	4.037	3.117
Throughput (GOPs)	15.75 (1.00×)	15.28 (0.97×)	14.84 (0.94×)	44.49 (2.82×)	498.49 (31.64×)
Area Efficiency (GOPs/mm ²)	12.54 (1.00×)	10.38 (0.83×)	12.25 (0.98×)	28.12 (2.24×)	352.54 (28.10×)
Energy Efficiency (GOPs/W)	9.56 (1.00×)	5.58 (0.58×)	5.70 (0.60×)	11.02 (1.15×)	159.94 (16.72×)

Buffer: 528 KB; DRAM Bandwidth: 64 GB/s.

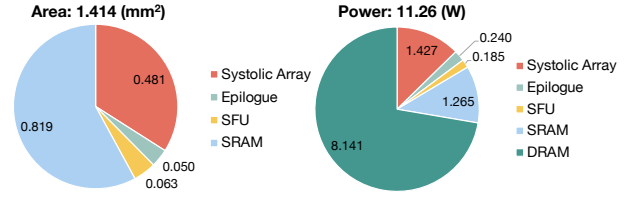


Fig. 9. *EVA* area and power breakdown.

does not rely on any particular method and can benefit from future improvements, such as fine-tuning [42] or other emerging optimizations [35], [58], [74].

D. Area and Power Comparison

As shown in Tbl. VIII, we compare the area and power of *EVA* and the baseline architectures. For a fair comparison, all designs are configured with the same minimum number of PEs. For *EVA*, compared to the INT8 systolic array, the increases in area and power are primarily due to its mixed-precision support for FP16 computation during the decoding stage. Although *EVA* does not have the smallest area or power consumption, it achieves significantly higher throughput and demonstrates superior energy and area efficiency using 2-bit VQ, highlighting the effectiveness of the proposed design.

Breakdown. Fig. 9 illustrates the area and power breakdown of *EVA*. On-chip SRAM occupies the largest area and has the second-highest power consumption, mainly due to double buffering and the frequent accesses to the output-codebook buffer. Off-chip DRAM accounts for the highest power consumption, which is consistent with the memory-bound nature of the decoding phase. Within the compute core, the PE array occupies the main area and power consumption, while the Epilogue Units introduce only a small overhead.

E. *EVA* Performance and Energy

Latency and energy on single-batch decoding. We evaluate *EVA* and the baselines on the fully connected (FC) layers of the LLaMA models during the decoding phase with a batch size of 1. As shown in Fig. 10 (a), all baselines suffer from low array utilization because decoding is executed as GEMV, which leads to long latency. On the 32×32 systolic array,

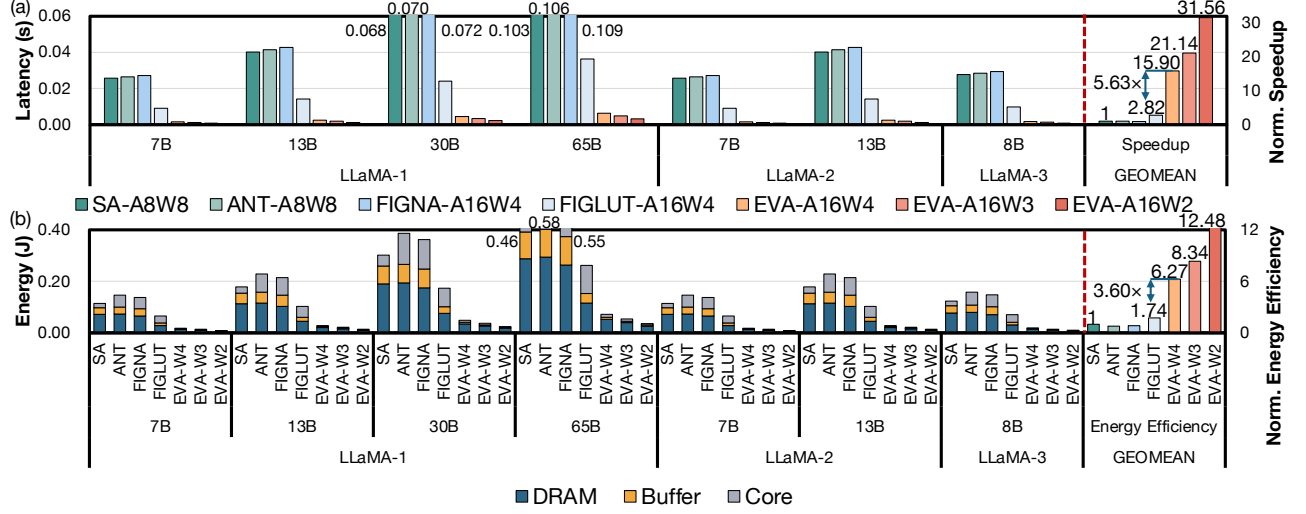


Fig. 10. Latency and energy consumption of the EVA and baseline accelerators on the fully connected layer with batch size=1 during the decoding phase of the LLaMA models. Method-*AnWm* denotes n-bit activation and m-bit weight.

only one lane is effectively active when batch size = 1. The utilization rates of ANT and FIGNA are further reduced due to the increased pipeline fill and drain overhead. As a LUT-based method, FIGLUT attains higher utilization by using 4-input LUTs and broadcasting their outputs to multiple PEs. However, its utilization rate remains relatively low (4.34%).

In contrast, *EVA-A16W2* achieves higher hardware utilization by transforming GEMV into GEMM with a conflict-free output LUT. Therefore, *EVA-A16W2* achieves speedups of $31.56\times$, $32.53\times$, $33.50\times$, and $11.17\times$ over SA, ANT, FIGNA, and FIGLUT, respectively. Furthermore, *EVA-A16W2* delivers $1.99\times$ and $1.49\times$ speedups over *EVA-A16W4* and *EVA-A16W3*, respectively. This improvement arises from the fact that lower-bit VQ uses fewer codebooks [15], [58], which proportionally reduces the associated computational cost.

Fig. 10 (b) reports the energy consumption. For all architectures, DRAM access dominates the total energy cost. Compared with SA, ANT, FIGNA, and FIGLUT, *EVA-A16W2* achieves $12.48\times$, $15.96\times$, $14.96\times$, and $7.17\times$ higher energy efficiency, respectively. *EVA-A16W2* further improves energy efficiency by $1.99\times$ and $1.50\times$ over *EVA-A16W4* and *EVA-A16W3*, respectively.

Effect of batch scaling. We assess the effect of multi-batch execution during the decoding phase on EVA and the baseline accelerators using the LLaMA-2-7B model. Here, *EVA-A8W8* refers to the INT8 computation method described in Sec. IV-B.

As shown in Fig. 11 (a), the latency for SA, ANT, FIGNA, FIGLUT, and *EVA-A8W8* increases slowly when the batch size is less than 8. This is because their hardware utilization remains low at smaller batch sizes, so the extra computation is partly hidden by improved array utilization. In contrast, *EVA-A16W4*, *EVA-A16W3*, and *EVA-A16W2* have already achieved high utilization when the batch size is small, leading to a nearly linear increase in latency as the batch size grows.

Fig. 11 (b) shows how energy consumption varies with batch

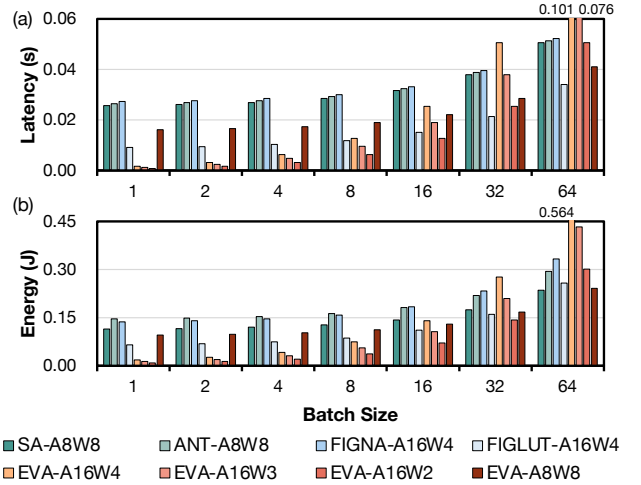


Fig. 11. Effect of batch scaling on LLaMA-2-7B.

size. The energy consumption of FIGNA increases faster than that of ANT. This is primarily due to increased overhead in DRAM accesses. We observe that when the batch size is 1, DRAM access accounts for 48.13% of the total energy in FIGNA and 50.27% in ANT. When the batch size becomes 64, these shares increase to 58.28% and 51.75%, respectively. This difference arises because FIGNA uses FP16 activations, which require about twice the memory bandwidth of ANT's 8-bit activations. The same effect is observed between FIGLUT and *EVA-A8W8*.

When the batch size is larger than 32, the latency and energy of *EVA-A16W2* become higher than *EVA-A8W8*. The same trend holds for *EVA-A16W4* and *EVA-A16W3*. As the batch size continues to increase, the workload changes from GEMV-style to GEMM-style computation, and the array utilization of all non-VQ architectures approaches nearly 100%. In this regime, the latencies of different architectures become similar,

TABLE IX
INPUT AND OUTPUT LENGTHS ACROSS MODELS AND DATASETS.

Model	LLaMA2-7B	Mixtral-8x7B	Qwen3-30B-A3B
Dataset	Dolly	Arxiv	GSM8K
Input Length	22.25	8575.45	66.03
Output Length	246.87	227.08	126.79
		8050.69	208.57
			61.51
			121.03

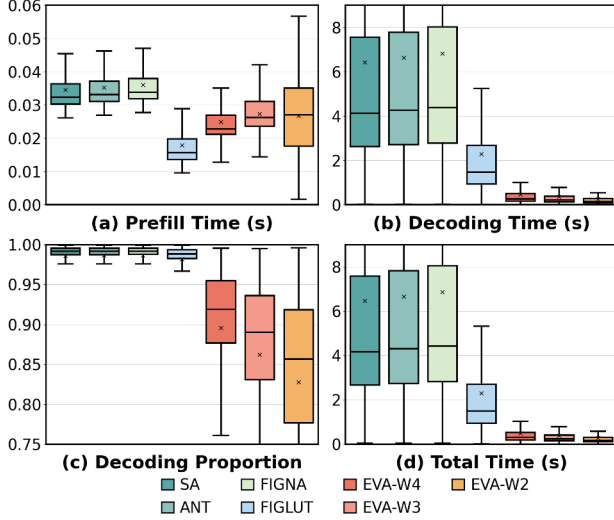


Fig. 12. Results of running LLaMA-2-7B on the Dolly dataset using EVA and baseline accelerators. (a) Distribution of prefill time; (b) Distribution of decoding time; (c) Distribution of the ratio of decoding time to total time; (d) Distribution of the total end-to-end execution time.

and the energy consumption mainly reflects their power.

End-to-end performance on real-world datasets. We evaluate the end-to-end performance of EVA and the baselines using LLaMA-2-7B model on the Dolly creative writing dataset [12]. Additionally, we evaluate the end-to-end performance of Mixtral-8x7B [29] and Qwen3-30B-A3B [69] on the Arxiv Summarization [11] and GSM8K [10] datasets. The input and output lengths for each dataset are detailed in Tbl. IX.

Following Fig. 11, for EVA-A16W4/3/2, we use the 32×32 PE array to compute INT8 results when the input length exceeds 8, 16, or 32 tokens. The results in Fig. 12 (a) show that all architectures have similar overall prefill latency. In Fig. 12 (b), EVA-A16W2 again demonstrates clear decoding speed advantages, with $17.06\times$ averaged speedup compared with all SOTA baselines. The Dolly dataset is decode-heavy. Therefore, decoding accounts for over 80% of the total execution time across all architectures, as shown in Fig. 12 (c). Consequently, EVA-A16W2 achieves an average $8.20\times$ to $24.49\times$ end-to-end speedup over the baselines, as illustrated in Fig. 12 (d). These results highlight the importance of designing accelerators that efficiently support both prefill and decoding phases.

In MoE models (Fig. 13), on the prefill-heavy Arxiv dataset, EVA-A16W2 achieves a $1.13\times$ – $2.28\times$ speedup over baselines. On decode-heavy GSM8K dataset, EVA significantly accelerates decoding, achieving a $5.01\times$ – $18.92\times$ speedup. Crucially,

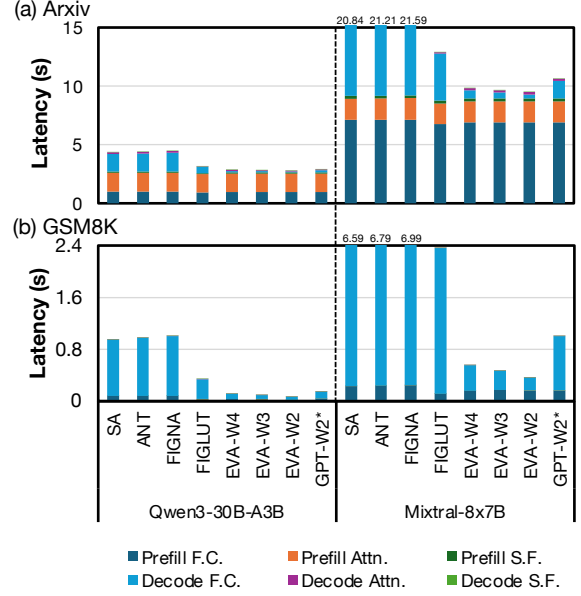


Fig. 13. End-to-end latency evaluation of MoE models on the (a) Arxiv Summarization and (b) GSM8K datasets. The results are categorized into prefill and decode phases for fully connected (F.C.), attention (Attn.), and special function (S.F.) layers. GPT-W2* represents the EVA architecture running the GPTVQ-4D algorithm.

the breakdown proves attention is not the bottleneck. While attention consumes up to 59.53% of runtime on the smaller Qwen model for Arxiv, this drops to 20.77% for Mixtral, and becomes negligible (0.07%–2.15%) on GSM8K. Special functions also account for just 0.08%–3.63% of latency due to execution overlap, validating our optimization focus on F.C. layers. Finally, although the GPTVQ-4D configuration ($2^n = 256, N = 256$) inherently limits EVA’s performance (Sec. VI-B), EVA running GPTVQ-4D still outperforms the SOTA FIGLUT baseline by $1.15\times$ on Arxiv and $2.31\times$ on GSM8K, confirming its effective support for diverse VQ algorithms.

F. Discussion

EVA’s Output Codebook Advantages. We examine EVA’s architectural advantages within a 32×8 FP16 output-stationary systolic array configured for VQ ($d = 8, n = 8, C = 1$), as illustrated in Table X. Standard dequantization retrieves vectors from codebook buffers divided into four banks to meet throughput requirements. However, simultaneous accesses to the same bank stall the array, introducing a $2.06\times$ latency overhead. To address this, we adapted the frequency-based replication of “hot” weight indices from the GPU-optimized VQ-LLM framework [41] into our simulator, which mitigates SRAM bank conflicts and yields a $1.74\times$ acceleration. In contrast, EVA fundamentally resolves this bottleneck by transforming weight-codebook lookups into output-codebook lookups. Because one Epilogue Unit (EU) accumulation implicitly replaces 8 MAC operations, this dimension collapse ($d = 8 \rightarrow 1$) reduces both SRAM storage and bandwidth

TABLE X
SCALING PERFORMANCE, AREA OVERHEAD, AND CONFLICT MITIGATION OF *EVA* CONFIGURATIONS ON LLAMA-2-7B.

Configuration\Method	VQ w. Conflict	VQ-LLM	VQ w/o. Conflict	<i>EVA</i> EU-4 × 1	<i>EVA</i> EU-32 × 1	<i>EVA</i> EU-32 × 4
Codebook SRAM Size	$8 \times 256 \times$ FP16 = 4KB	$8 \times 256 \times$ FP16 × 2.5 = 10KB	$8 \times 256 \times$ FP16 × 4 = 16KB	$1 \times 256 \times$ FP16 × 4 = 2KB	$1 \times 256 \times$ FP16 × 32 = 16KB	$1 \times 256 \times$ FP16 × 32 × 4 = 64KB
Codebook SRAM Bandwidth	4 bank × 8 × FP16 = 64 B/Cycle	4 bank × 8 × FP16 = 64 B/Cycle	4 bank × 8 × FP16 = 64 B/Cycle	4 bank × 1 × FP16 = 8 B/Cycle	32 bank × 1 × FP16 = 64 B/Cycle	32 bank × 1 × FP16 × 4 = 256 B/Cycle
Systolic Array Size	32×8 (FP16)	32×8 (FP16)	32×8 (FP16)	32×8 (FP16)	32×8 (FP16)	32×8 (FP16)
Epilogue Unit Size	-	-	-	4-input Adder	32-input Adder	4 × 32-input Adders
Normalized Array Area	1.00×	1.00×	1.00×	1.01×	1.05×	1.18×
Normalized Speedup	1.00×	1.74×	2.06×	2.12×	16.95×	64.84×
Note	Full Conflicts	50% Conflicts	No Conflicts	No Conflicts	No Conflicts	No Conflicts

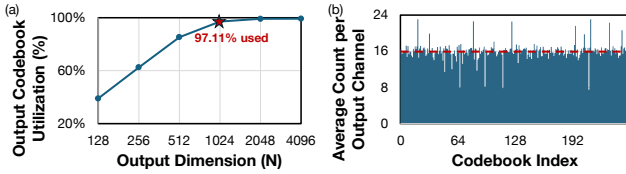


Fig. 14. Evaluation of spurious computations in the proposed method. (a) The effective computation rate (percentage of accessed codebook entries) scales with the number of output channels (N); (b) The access frequency of codebook indices averaged by output channel.

demands by $8\times$ under an identical 4-bank configuration. The final two columns of Table X demonstrate *EVA*’s scalability.

Analysis of Spurious Computations. In *EVA*, a multiplication is only “spurious” if a centroid is computed but never referenced by any output channel N . While redundancies can occur if the codebook size 2^n exceeds N , typical LLM layers feature $N \gg 2^n$. Furthermore, the optimal VQ algorithm maximizes information entropy, naturally driving a uniform distribution of weight indices to prevent centroid collapse, as verified in Fig. 14(b). Under this uniform distribution, the expected number of utilized centroids is $\mathbb{E}[U] = 2^n \left[1 - \left(1 - \frac{1}{2^n}\right)^N\right]$. For $2^n = 256$ and $N = 1024$, the theoretical codebook utilization rate is 98.2%, which closely matches our observed 97.11% in Fig. 14(a). Therefore, centroid usage is inherently well-balanced, and the overhead from spurious multiplications is negligible.

VII. RELATED WORK

Algorithm-level optimization for VQ-style LLM inference. Recent studies accelerate LLMs by leveraging VQ or similar methods. VQ-LLM [41] improves GPU execution of vector-quantized LLMs by identifying and optimizing frequently accessed codebook entries. CodeGEMM [47] demonstrates the feasibility of VQ-based computation by reformulating quantized matrix operations for efficient GPU execution. MADDNESS [5] proposes a hashing-based approximate matrix multiplication algorithm that replaces multiplications with learned lookup tables. While these methods explore lookup-

based computation in software, *EVA* co-designs the algorithm and architecture for LLM decoding.

Lookup-based LLM accelerators. Several accelerators eliminate MAC operations through lookup-table computation. FIGLUT [48], LUT Tensor Cores [44], and Platinum [53] build lookup tables from activations and use weights as indices to skip multiplications in GEMM. Recent result-reuse accelerators, Prosperity [65], Transitive Array [20], Phi [64], and Focus [66], enable lookup-based result reuse through activation sparsity. While these designs eliminate arithmetic operations through lookup tables, they do not address the memory-system bottlenecks of VQ-based LLM inference; in contrast, *EVA* resolves memory bank conflicts to maximize compute utilization and is, to our knowledge, the first architecture-level accelerator for vector-quantized LLM inference.

VIII. CONCLUSION

We presented *EVA*, a VQ-based architecture that restructures LLM decoding around a codebook-driven GEMM formulation. By directly multiplying inputs with the weight codebook and performing conflict-free lookups on an output codebook, *EVA* converts memory-bound GEMV into compute-efficient GEMM while eliminating codebook access conflicts. A mixed-precision GEMM unit and lightweight epilogue enable FP16 VQ decoding and INT8 prefill on a shared systolic array, achieving high utilization with modest hardware overhead. Evaluations on LLaMA-family models show that *EVA* sustains competitive accuracy at low bit-widths and delivers up to $11.17\times$ speedup and $7.17\times$ higher energy efficiency over the lookup-based baseline. Our work highlights the potential of codebook-aware, algorithm–hardware co-design for efficient LLM decoding.

IX. ACKNOWLEDGMENT

This work was supported in part by NSF-2112562 and ARO W911NF-23-2-0224. The authors sincerely thank the anonymous reviewers for their constructive feedback and valuable suggestions that greatly improved the quality of this work. The authors also express their gratitude to Yuzhou Chen for his technical support and insightful discussions.

REFERENCES

- [1] A. Agrawal, A. Panwar, J. Mohan, N. Kwatra, B. S. Gulavani, and R. Ramjee, "Sarathi: Efficient llm inference by piggybacking decodes with chunked prefills," *arXiv preprint arXiv:2308.16369*, 2023.
- [2] J. Bai, S. Bai, Y. Chu, Z. Cui, K. Dang, X. Deng, Y. Fan, W. Ge, Y. Han, F. Huang *et al.*, "Qwen technical report," *arXiv preprint arXiv:2309.16609*, 2023.
- [3] R. Balasubramanian, A. B. Kahng, N. Muralimanohar, A. Shafiee, and V. Srinivas, "Cacti 7: New tools for interconnect exploration in innovative off-chip memories," *ACM Transactions on Architecture and Code Optimization (TACO)*, vol. 14, no. 2, pp. 1–25, 2017.
- [4] Y. Bisk, R. Zellers, J. Gao, Y. Choi *et al.*, "Piqa: Reasoning about physical commonsense in natural language," in *Proceedings of the AAAI conference on artificial intelligence*, vol. 34, no. 05, 2020, pp. 7432–7439.
- [5] D. Blalock and J. Gutttag, "Multiplying matrices without multiplying," in *International Conference on Machine Learning*. PMLR, 2021, pp. 992–1004.
- [6] F. Cheng, C. Guo, C. Wei, J. Zhang, C. Zhou, E. Hanson, J. Zhang, X. Liu, H. Li, and Y. Chen, "Ecco: Improving memory bandwidth and capacity for llms via entropy-aware cache compression," in *Proceedings of the 52nd Annual International Symposium on Computer Architecture*, 2025, pp. 793–807.
- [7] J. Choquette, W. Gandhi, O. Giroux, N. Stam, and R. Krashinsky, "Nvidia a100 tensor core gpu: Performance and innovation," *IEEE Micro*, vol. 41, no. 2, pp. 29–35, 2021.
- [8] C. Clark, K. Lee, M.-W. Chang, T. Kwiatkowski, M. Collins, and K. Toutanova, "Boolq: Exploring the surprising difficulty of natural yes/no questions," in *Proceedings of the 2019 conference of the north American chapter of the association for computational linguistics: Human language technologies, volume 1 (long and short papers)*, 2019, pp. 2924–2936.
- [9] P. Clark, I. Cowhey, O. Etzioni, T. Khot, A. Sabharwal, C. Schoenick, and O. Tafjord, "Think you have solved question answering? try arc, the ai2 reasoning challenge," *arXiv preprint arXiv:1803.05457*, 2018.
- [10] K. Cobbe, V. Kosaraju, M. Bavarian, M. Chen, H. Jun, L. Kaiser, M. Plappert, J. Tworek, J. Hilton, R. Nakano *et al.*, "Training verifiers to solve math word problems," *arXiv preprint arXiv:2110.14168*, 2021.
- [11] A. Cohan, F. Dernoncourt, D. S. Kim, T. Bui, S. Kim, W. Chang, and N. Goharian, "A discourse-aware attention model for abstractive summarization of long documents," in *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 2 (Short Papers)*, 2018, pp. 615–621.
- [12] M. Conover, M. Hayes, A. Mathur, J. Xie, J. Wan, S. Shah, A. Ghodsi, P. Wendell, M. Zaharia, and R. Xin. (2023) Free dolly: Introducing the world's first truly open instruction-tuned llm. [Online]. Available: <https://www.databricks.com/blog/2023/04/12/dolly-first-open-commercially-viable-instruction-tuned-llm>
- [13] DeepSeek-AI, "Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning," *CoRR*, vol. abs/2501.12948, 2025. [Online]. Available: <https://doi.org/10.48550/arXiv.2501.12948>
- [14] J. Devlin, M. Chang, K. Lee, and K. Toutanova, "BERT: pre-training of deep bidirectional transformers for language understanding," *CoRR*, vol. abs/1810.04805, 2018. [Online]. Available: <http://arxiv.org/abs/1810.04805>
- [15] V. Egiazarian, A. Panferov, D. Kuznedelev, E. Frantar, A. Babenko, and D. Alistarh, "Extreme compression of large language models via additive quantization," 2024.
- [16] E. Frantar, S. Ashkboos, T. Hoefler, and D. Alistarh, "GPTQ: accurate post-training quantization for generative pre-trained transformers," *CoRR*, vol. abs/2210.17323, 2022. [Online]. Available: <https://doi.org/10.48550/arXiv.2210.17323>
- [17] GitHub, "Github copilot," <https://github.com/features/copilot>, 2025, accessed: 2025-11-17.
- [18] D. Gou, S. Byun, N. Malpeddi, G. De Micheli, P. Vaste, J. Song, and W. S. Chung, "Carqv: Corrective adaptor with group residual vector quantization for llm embedding compression," in *Findings of the Association for Computational Linguistics: EMNLP 2025*, 2025, pp. 18 594–18 604.
- [19] C. Guo, J. Tang, W. Hu, J. Leng, C. Zhang, F. Yang, Y. Liu, M. Guo, and Y. Zhu, "Olive: Accelerating large language models via hardware-friendly outlier-victim pair quantization," in *Proceedings of the 50th Annual International Symposium on Computer Architecture*, 2023, pp. 1–15.
- [20] C. Guo, C. Wei, J. Tang, B. Duan, S. Han, H. Li, and Y. Chen, "Transitive array: An efficient gemm accelerator with result reuse," in *Proceedings of the 52nd Annual International Symposium on Computer Architecture*, 2025, pp. 990–1004.
- [21] C. Guo, C. Zhang, J. Leng, Z. Liu, F. Yang, Y. Liu, M. Guo, and Y. Zhu, "Ant: Exploiting adaptive numerical data type for low-bit deep neural network quantization," in *2022 55th IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 2022, pp. 1414–1433.
- [22] S. Han, H. Mao, and W. J. Dally, "Deep compression: Compressing deep neural networks with pruning, trained quantization and huffman coding," *arXiv preprint arXiv:1510.00149*, 2015.
- [23] W. Hu, H. Zhang, C. Guo, Y. Feng, R. Guan, Z. Hua, Z. Liu, Y. Guan, M. Guo, and J. Leng, "M-ant: Efficient low-bit group quantization for llms via mathematically adaptive numerical type," in *2025 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 2025, pp. 1112–1126.
- [24] X. Hu, Y. Cheng, D. Yang, Z. Yuan, J. Yu, C. Xu, and S. Zhou, "I-llm: Efficient integer-only inference for fully-quantized low-bit large language models," *arXiv preprint arXiv:2405.17849*, 2024.
- [25] I. A. Huijben, M. Douze, M. Muckley, R. J. Van Sloun, and J. Verbeek, "Residual quantization with implicit neural codebooks," *arXiv preprint arXiv:2401.14732*, 2024.
- [26] B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam, and D. Kalenichenko, "Quantization and training of neural networks for efficient integer-arithmetic-only inference," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2018, pp. 2704–2713.
- [27] J. Jang, Y. Kim, J. Lee, and J.-J. Kim, "Figma: Integer unit-based accelerator design for fp-int gemm preserving numerical accuracy," in *2024 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 2024, pp. 760–773.
- [28] A. Q. Jiang, A. Sablayrolles, A. Mensch, C. Bamford, D. S. Chaplot, D. de Las Casas, F. Bressand, G. Lengyel, G. Lample, L. Saulnier, L. R. Lavaud, M. Lachaux, P. Stock, T. L. Scao, T. Lavril, T. Wang, T. Lacroix, and W. E. Sayed, "Mistral 7b," *CoRR*, vol. abs/2310.06825, 2023. [Online]. Available: <https://doi.org/10.48550/arXiv.2310.06825>
- [29] A. Q. Jiang, A. Sablayrolles, A. Roux, A. Mensch, B. Savary, C. Bamford, D. S. Chaplot, D. d. l. Casas, E. B. Hanna, F. Bressand *et al.*, "Mixtral of experts," *arXiv preprint arXiv:2401.04088*, 2024.
- [30] N. P. Jouppi, C. Young, N. Patil, D. Patterson, G. Agrawal, R. Bajwa, S. Bates, S. Bhatia, N. Boden, A. Borchers *et al.*, "In-datacenter performance analysis of a tensor processing unit," in *Proceedings of the 44th annual international symposium on computer architecture*, 2017, pp. 1–12.
- [31] S. Kim, A. Gholami, Z. Yao, M. W. Mahoney, and K. Keutzer, "I-bert: Integer-only bert quantization," in *International conference on machine learning*. PMLR, 2021, pp. 5506–5518.
- [32] S. Kim, C. Hooper, A. Gholami, Z. Dong, X. Li, S. Shen, M. W. Mahoney, and K. Keutzer, "Squeezellm: dense-and-sparse quantization," in *Proceedings of the 41st International Conference on Machine Learning*, ser. ICML'24. JMLR.org, 2024.
- [33] S. Lee, S.-t. Woo, J.-g. Jin, C. Lee, and E. Park, "Amq: Enabling autolm for mixed-precision weight-only quantization of large language models," in *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, 2025, pp. 35 520–35 538.
- [34] G. Li, S. Ye, C. Chen, Y. Wang, F. Yang, T. Cao, C. Liu, M. M. S. Aly, and M. Yang, "Lut-dla: Lookup table as efficient extreme low-bit deep learning accelerator," in *2025 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 2025, pp. 671–684.
- [35] J. Li, Y. Zhang, M. Y. Hassan, T. Chafekar, T. Cai, Z. Ren, P. Guo, F. Karimzadeh, C. Wang, and C. Gan, "Commvq: Commutative vector quantization for kv cache compression," *arXiv preprint arXiv:2506.18879*, 2025.
- [36] S. Li, Z. Yang, D. Reddy, A. Srivastava, and B. L. Jacob, "Dramsim3: A cycle-accurate, thermal-capable DRAM simulator," *IEEE Comput. Archit. Lett.*, vol. 19, no. 2, pp. 110–113, 2020. [Online]. Available: <https://doi.org/10.1109/LCA.2020.2973991>
- [37] J. Lin, J. Tang, H. Tang, S. Yang, W.-M. Chen, W.-C. Wang, G. Xiao, X. Dang, C. Gan, and S. Han, "Awq: Activation-aware weight quanti-

- zation for on-device llm compression and acceleration,” *Proceedings of machine learning and systems*, vol. 6, pp. 87–100, 2024.
- [38] Y. Lin, Y. Fu, J. Zhang, Y. Liu, J. Zhang, J. Sun, H. Li, Y. Chen *et al.*, “Speechprune: Context-aware token pruning for speech information retrieval,” *arXiv preprint arXiv:2412.12009*, 2024.
- [39] Y. Lin, H. Tang, S. Yang, Z. Zhang, G. Xiao, C. Gan, and S. Han, “Qserve: W4a8kv4 quantization and system co-design for efficient llm serving,” *arXiv preprint arXiv:2405.04532*, 2024.
- [40] Y. Liu, J. Wen, Y. Wang, S. Ye, L. L. Zhang, T. Cao, C. Li, and M. Yang, “Vptq: Extreme low-bit vector post-training quantization for large language models,” *arXiv preprint arXiv:2409.17066*, 2024.
- [41] Z. Liu, X. Luo, J. Guo, W. Ni, Y. Zhou, Y. Guan, C. Guo, W. Cui, Y. Feng, M. Guo *et al.*, “Vq-llm: High-performance code generation for vector quantization augmented llm inference,” in *2025 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 2025, pp. 1496–1509.
- [42] V. Malinovskii, D. Mazur, I. Ilin, D. Kuznedelev, K. Burlachenko, K. Yi, D. Alistarh, and P. Richtarik, “Pv-tuning: Beyond straight-through estimation for extreme llm compression,” 2024.
- [43] S. Merity, C. Xiong, J. Bradbury, and R. Socher, “Pointer sentinel mixture models,” *arXiv preprint arXiv:1609.07843*, 2016.
- [44] Z. Mo, L. Wang, J. Wei, Z. Zeng, S. Cao, L. Ma, N. Jing, T. Cao, J. Xue, F. Yang, and M. Yang, “LUT tensor core: Lookup table enables efficient low-bit LLM inference acceleration,” *CoRR*, vol. abs/2408.06003, 2024. [Online]. Available: <https://doi.org/10.48550/arXiv.2408.06003>
- [45] M. Navardi, R. Aalishah, Y. Fu, Y. Lin, H. Li, Y. Chen, and T. Mohsenin, “Genai at the edge: Comprehensive survey on empowering edge devices,” in *Proceedings of the AAAI Symposium Series*, vol. 5, no. 1, 2025, pp. 180–187.
- [46] OpenAI, “Gpt-4 technical report,” <https://cdn.openai.com/papers/gpt-4.pdf>, 2023, accessed: 2025-11-16.
- [47] G. Park, J. Bae, B. Kim, J. Ryu, H. Kim, S. J. Kwon, D. Lee *et al.*, “Codegemm: A codebook-centric approach to efficient gemm in quantized llms,” *arXiv preprint arXiv:2512.17970*, 2025.
- [48] G. Park, H. Kwon, J. Kim, J. Bae, B. Park, D. Lee, and Y. Lee, “Figlut: An energy-efficient accelerator design for fp-int gemm using look-up tables,” in *2025 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 2025, pp. 1098–1111.
- [49] P. Patel, E. Choukse, C. Zhang, A. Shah, I. Goiri, S. Maleki, and R. Bianchini, “Splitwise: Efficient generative LLM inference using phase splitting,” in *51st ACM/IEEE Annual International Symposium on Computer Architecture, ISCA 2024, Buenos Aires, Argentina, June 29 - July 3, 2024*. IEEE, 2024, pp. 118–132. [Online]. Available: <https://doi.org/10.1109/ISCA59077.2024.00019>
- [50] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever, “Language models are unsupervised multitask learners,” 2019.
- [51] M. Roemmele, C. A. Bejan, and A. S. Gordon, “Choice of plausible alternatives: An evaluation of commonsense causal reasoning,” in *AAAI spring symposium: logical formalizations of commonsense reasoning*, 2011, pp. 90–95.
- [52] K. Sakaguchi, R. L. Bras, C. Bhagavatula, and Y. Choi, “Winogrande: An adversarial winograd schema challenge at scale,” *Communications of the ACM*, vol. 64, no. 9, pp. 99–106, 2021.
- [53] H. Shan, C. Guo, C. Wei, F. Cheng, J. Zhang, H. H. Li, and Y. Chen, “Platinum: Path-adaptable lut-based accelerator tailored for low-bit weight matrix multiplication,” in *2026 31st Asia and South Pacific Design Automation Conference (ASP-DAC)*. IEEE, 2026, pp. 1449–1455.
- [54] L. Team, “The llama 3 herd of models,” *CoRR*, vol. abs/2407.21783, 2024. [Online]. Available: <https://doi.org/10.48550/arXiv.2407.21783>
- [55] S. Thakur, B. Ahmad, H. Pearce, B. Tan, B. Dolan-Gavitt, R. Karri, and S. Garg, “Verigen: A large language model for verilog code generation,” *ACM Trans. Des. Autom. Electron. Syst.*, vol. 29, no. 3, Apr. 2024. [Online]. Available: <https://doi.org/10.1145/3643681>
- [56] H. Touvron and *et al.*, “Llama 2: Open foundation and fine-tuned chat models,” *CoRR*, vol. abs/2307.09288, 2023. [Online]. Available: <https://doi.org/10.48550/arXiv.2307.09288>
- [57] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. Rozière, N. Goyal, E. Hambro, F. Azhar *et al.*, “Llama: Open and efficient foundation language models,” *arXiv preprint arXiv:2302.13971*, 2023.
- [58] A. Tseng, J. Chee, Q. Sun, V. Kuleshov, and C. De Sa, “Quip#: Even better llm quantization with hadamard incoherence and lattice codebooks,” *arXiv preprint arXiv:2402.04396*, 2024.
- [59] M. Van Baalen, A. Kuzmin, I. Koryakovskiy, M. Nagel, P. Couperus, C. Bastoul, E. Mahurin, T. Blankevoort, and P. Whatmough, “Gptvq: The blessing of dimensionality for llm quantization,” *arXiv preprint arXiv:2402.15319*, 2024.
- [60] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, “Attention is all you need,” in *Advances in Neural Information Processing Systems 30: Annual Conference on Neural Information Processing Systems 2017, December 4-9, 2017, Long Beach, CA, USA*, I. Guyon, U. von Luxburg, S. Bengio, H. M. Wallach, R. Fergus, S. V. N. Vishwanathan, and R. Garnett, Eds., 2017, pp. 5998–6008. [Online]. Available: <https://proceedings.neurips.cc/paper/2017/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html>
- [61] H. Wang, S. Ma, L. Dong, S. Huang, H. Wang, L. Ma, F. Yang, R. Wang, Y. Wu, and F. Wei, “Bitnet: Scaling 1-bit transformers for large language models,” *arXiv preprint arXiv:2310.11453*, 2023.
- [62] Q. Wang*, J. Ke*, M. Tomizuka, K. Keutzer, and C. Xu, “Dobi-svd: Differentiable svd for llm compression and some new perspectives,” in *The Thirteenth International Conference on Learning Representations*, 2025.
- [63] Q. Wang, J. Ke, H. Ye, Y. Lin, Y. Fu, J. Zhang, K. Keutzer, C. Xu, and Y. Chen, “Angles don’t lie: Unlocking training-efficient rl through the model’s own signals,” *arXiv preprint arXiv:2506.02281*, 2025.
- [64] C. Wei, B. Duan, C. Guo, J. Zhang, Q. Song, H. Li, and Y. Chen, “Phi: Leveraging pattern-based hierarchical sparsity for high-efficiency spiking neural networks,” in *Proceedings of the 52nd Annual International Symposium on Computer Architecture*, 2025, pp. 930–943.
- [65] C. Wei, C. Guo, F. Cheng, S. Li, H. F. Yang, H. H. Li, and Y. Chen, “Prosperity: Accelerating spiking neural networks via product sparsity,” in *2025 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 2025, pp. 806–820.
- [66] C. Wei, C. Guo, J. Zhang, H. Shan, Y. Xu, Z. Zhang, Y. Liu, Q. Wang, C. Zhou, H. H. Li *et al.*, “Focus: A streaming concentration architecture for efficient vision-language models,” in *2026 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 2026, pp. 1–18.
- [67] G. Xiao, J. Lin, M. Seznec, H. Wu, J. Demouth, and S. Han, “Smoothquant: Accurate and efficient post-training quantization for large language models,” in *International Conference on Machine Learning, ICML 2023, 23-29 July 2023, Honolulu, Hawaii, USA*, ser. Proceedings of Machine Learning Research, A. Krause, E. Brunskill, K. Cho, B. Engelhardt, S. Sabato, and J. Scarlett, Eds., vol. 202. PMLR, 2023, pp. 38 087–38 099. [Online]. Available: <https://proceedings.mlr.press/v202/xiao23c.html>
- [68] C. Xu, Y. Wu, X. Yang, B. Chen, M. Lentz, D. Zhuo, and L. W. Wills, “Llm. 265: Video codecs are secretly tensor codecs,” in *Proceedings of the 58th IEEE/ACM International Symposium on Microarchitecture@*, 2025, pp. 445–460.
- [69] A. Yang, A. Li, B. Yang, B. Zhang, B. Hui, B. Zheng, B. Yu, C. Gao, C. Huang, C. Lv *et al.*, “Qwen3 technical report,” *arXiv preprint arXiv:2505.09388*, 2025.
- [70] Z. Yao, R. Yazdani Aminabadi, M. Zhang, X. Wu, C. Li, and Y. He, “Zeroquant: Efficient and affordable post-training quantization for large-scale transformers,” *Advances in neural information processing systems*, vol. 35, pp. 27 168–27 183, 2022.
- [71] H. You, Y. Guo, Y. Fu, W. Zhou, H. Shi, X. Zhang, S. Kundu, A. Yazdanbakhsh, and Y. C. Lin, “Shiftaddllm: Accelerating pretrained llms via post-training multiplication-less reparameterization,” *Advances in Neural Information Processing Systems*, vol. 37, pp. 24 822–24 848, 2024.
- [72] G. Yu, J. S. Jeong, G. Kim, S. Kim, and B. Chun, “Orca: A distributed serving system for transformer-based generative models,” in *16th USENIX Symposium on Operating Systems Design and Implementation, OSDI 2022, Carlsbad, CA, USA, July 11-13, 2022*, M. K. Aguilera and H. Weatherspoon, Eds. USENIX Association, 2022, pp. 521–538. [Online]. Available: <https://www.usenix.org/conference/osdi22/presentation/you>
- [73] A. H. Zadeh, I. Edo, O. M. Awad, and A. Moshovos, “Gobo: Quantizing attention-based nlp models for low latency and energy efficient inference,” in *2020 53rd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 2020, pp. 811–824.
- [74] T. Zhang, J. Yi, Z. Xu, and A. Shrivastava, “Kv cache is 1 bit per channel: Efficient large language model inference with coupled quantization, 2024b,” URL <https://arxiv.org/abs/2405.03917>.

A. Abstract

Our artifact contains (1) a hardware simulator that reproduces all hardware evaluation results (Figures 8–14 and Tables III, VIII–IX) from the *EVA* paper, and (2) an algorithm evaluation that reproduces the accuracy tables (Tables V–VII) using pre-trained AQLM-quantized model checkpoints.

The hardware simulator models latency, energy, power, and area for *EVA* and baseline architectures (SA, ANT, FIGNA, FIGLUT) across dense and Mixture-of-Experts (MoE) LLMs. The algorithm evaluation runs perplexity and downstream benchmark evaluations on quantized LLMs hosted on Hugging Face. The artifact includes the full simulation pipeline, YAML-based study configurations, pre-processed trace files, evaluation scripts migrated from the AQLM repository [15], parallel shell scripts for batch reproduction, and Jupyter notebooks that render all paper-facing tables and figures.

B. Artifact check-list (meta-information)

- **Algorithm:** Hardware simulation of VQ-based *EVA* architecture for LLM inference; AQLM-based quantization evaluation.
- **Program:** Python (NumPy, pandas, PyTorch, Transformers).
- **Compilation:** Hardware simulator is interpreted in Python. Algorithm evaluation may JIT-compile AQLM CUDA/C++ extensions through `ninja`; GCC/G++ 11.x is recommended, and GCC/G++ 11.4.0 was tested.
- **Transformations:** N/A.
- **Binary:** N/A.
- **Model:** LLaMA-2-7B, LLaMA-2-13B [56], Mixtral-8x7B [29], Qwen3-30B-A3B [69].
- **Data set:** Dolly Creative Writing [12], Arxiv [11], GSM8K [10]; WikiText-2 dataset [43], PIQA [4], COPA [51], ARC-Easy, ARC-Challenge [9], BoolQ [8], and Winogrande [52].
- **Run-time environment:** Linux (tested on Ubuntu 20.04+ and Ubuntu 22.04), Python 3.11, Conda, GCC/G++ 11.x for AQLM extension builds.
- **Hardware:** Any x86-64 CPU with 16+ GB RAM (no GPU required for hardware simulation). Algorithm evaluation requires an NVIDIA GPU with ≥ 24 GB VRAM (A100-80GB recommended).
- **Run-time state:** Deterministic simulation (fixed seeds).
- **Execution:** Python CLI scripts, parallel shell scripts, and Jupyter notebooks.
- **Metrics:** Latency (cycles, seconds), energy (J), power (W), area (mm^2), throughput (GOPs), speedup; perplexity, accuracy (%).
- **Output:** CSV files containing per-study simulation results; JSON files with algorithm evaluation results; notebook-rendered tables and figures.
- **Experiments:** 9 hardware simulation studies reproducing Figs. 8–14 and Tables III, VIII–IX; 10 algorithm evaluations reproducing Tables V–VII.
- **How much disk space required (approximately)?:** ~ 10 GB for hardware simulation; ~ 100 GB additional for algorithm evaluation (model checkpoints).
- **How much time is needed to prepare workflow (approximately)?:** ~ 5 minutes (Conda environment setup and package installation).
- **How much time is needed to complete experiments (approximately)?:** ~ 2 hours for hardware simulation (Steps 1–9); ~ 6 – 8 hours additional for algorithm evaluation (Steps 10–12, on

A100-80GB, depending on checkpoint download speed, GPU occupancy, and CUDA extension build/cache state).

- **Publicly available?:** Yes. <https://github.com/dbw6/Eva.git>.
- **Code licenses (if publicly available)?:** MIT License.
- **Data licenses (if publicly available)?:** Hugging Face dataset licenses (Apache 2.0 for Dolly; original licenses for Arxiv and GSM8K).
- **Workflow automation framework used?:** N/A.
- **Archived (provide DOI)?:** <https://doi.org/10.5281/zenodo.19433707>.

C. Description

1) *How to access:* The source code is publicly available at: <https://github.com/dbw6/Eva.git>. The artifact sources are also archived at Zenodo: <https://doi.org/10.5281/zenodo.19433707>. Pretrained weights for all evaluated models (LLaMA-2-7B, LLaMA-2-13B, Mixtral-8x7B, and Qwen3-30B-A3B) and datasets are available at: <https://huggingface.co/collections/dbw6/eva>.

2) *Hardware dependencies:* **Hardware Simulator:** Any x86-64 machine with at least 16 GB of RAM and 10 GB of free disk space. No GPU is required; all simulations run on the CPU. Internet access is required for the first run to download Hugging Face models and datasets.

Algorithm Evaluation: An NVIDIA GPU with at least 24 GB VRAM (A100-80GB recommended), CUDA 12.x, and ~ 100 GB of additional disk space for downloading quantized model checkpoints.

3) *Software dependencies:*

- OS: Linux (tested on Ubuntu 20.04+)
- Python 3.11 and Conda (Miniconda or Anaconda)
- Compiler: GCC/G++ 11.x recommended; GCC/G++ 11.4.0 was used during artifact evaluation. AQLM may JIT-compile CUDA/C++ extensions through `ninja`, so the host compiler should be compatible with the installed CUDA toolkit.
- Core packages: `numpy`, `pandas`, `pyyaml`, `matplotlib`, `transformers`, `datasets`, `huggingface_hub`
- Quantization support: `aqlm[gpu,cpu]>=1.1.6`
- Evaluation packages: `torch>=2.3.0`, `accelerate>=0.29.3`, `safetensors>=0.4.0`, `sentencepiece`, and `lm-evaluation-harness`

D. Installation

Installation is performed via Conda and pip. The environment can be set up using the following commands:

```
conda create -n eva python=3.11 -y
conda activate eva
pip install -e .
pip install "aqlm[gpu,cpu]>=1.1.6"
pip install jupyter nbclient
```

For the algorithm evaluation, additional PyTorch and Transformer dependencies must be installed as detailed in the `Eva/algorithm/README.md`. If AQLM CUDA extension compilation fails, users should first verify that gcc

--version, g++ --version, and nvcc --version are mutually compatible with the selected PyTorch/CUDA installation.

E. Experiment workflow

The evaluation is split into two independent workflows:

1. Hardware Simulation (Steps 1–9): Nine hardware simulation studies can be executed either one at a time via the CLI or with the provided parallel runner:

```
scripts/run_simulator_parallel.sh
```

2. Algorithm Evaluation (Steps 10–12): Ten evaluations (4 perplexity + 6 downstream accuracy) can be run on a GPU to reproduce Tables V–VII using the `eval_ppl.py` and `lmeval.py` scripts. The provided multi-GPU runner distributes these jobs across a comma-separated GPU list and runs at most one evaluation per GPU:

```
scripts/run_algorithm_parallel.sh
```

F. Evaluation and expected results

Each hardware study produces CSV files under `simulator/output/`, while algorithm evaluations produce JSON files under `algorithm/output/`. The provided Jupyter notebooks render these raw data files into the exact tables and figures presented in the paper.

The total runtime for the hardware simulation is dominated by Step 9 (e2e), specifically the `fig13_moe` scenario which simulates MoE models (Mixtral-8x7B and Qwen3-30B-A3B) across datasets with 100 samples each. Steps 1–8 complete in under 6 minutes combined. The full end-to-end hardware simulation takes approximately 2 hours. The algorithm evaluation takes approximately 6–8 hours on an A100-80GB GPU for a clean full run; multi-GPU execution with `scripts/run_algorithm_parallel.sh` can reduce wall-clock time when multiple suitable GPUs are available.

G. Experiment customization

Users can customize experiments by editing the YAML configuration files under `simulator/configs/studies/`. Each study YAML specifies models, methods, sequence lengths, batch sizes, and study-specific parameters. The CLI also accepts overrides via command-line arguments (e.g., `--models`, `--methods`, `--scenarios`).

H. Methodology

Submission, reviewing and badging methodology:

- <https://www.acm.org/publications/policies/artifact-review-and-badging-current>
- <https://cTuning.org/ae>